

Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

Г.И. Ревунков, Ю.Е. Гапанюк, А.Н. Нардид

**ВВЕДЕНИЕ В ОБЪЕКТНО-ОРИЕНТИРОВАННУЮ И
ФУНКЦИОНАЛЬНУЮ ПАРАДИГМУ
ПРОГРАММИРОВАНИЯ НА ПРИМЕРЕ РЕШЕНИЯ
ЗАДАЧИ КВАДРАТНОГО УРАВНЕНИЯ**

Электронное учебное издание – рабочая версия

Москва, 2024

ОГЛАВЛЕНИЕ

1	ПРИМЕНЕНИЕ РАЗЛИЧНЫХ ПАРАДИГМ ПРОГРАММИРОВАНИЯ НА ПРИМЕРЕ РЕШЕНИЯ КВАДРАТНОГО УРАВНЕНИЯ.....	3
1.1	ПРИМЕР НА ЯЗЫКЕ C# С ИСПОЛЬЗОВАНИЕМ СПИСКА КОРНЕЙ	3
1.2	ПРИМЕР НА ЯЗЫКЕ C# С ИСПОЛЬЗОВАНИЕМ ПЕРЕЧИСЛЕНИЯ.....	4
1.3	ПРИМЕР НА ЯЗЫКЕ F# С ИСПОЛЬЗОВАНИЕМ АЛГЕБРАИЧЕСКОГО ТИПА.....	5
1.4	ПРИМЕР НА ЯЗЫКЕ C# С ИСПОЛЬЗОВАНИЕМ ИНТЕРФЕЙСА И НАСЛЕДУЕМЫХ КЛАССОВ.....	7
1.5	ПРИМЕР НА ЯЗЫКЕ F# С ИСПОЛЬЗОВАНИЕМ ИНТЕРФЕЙСА И НАСЛЕДУЕМЫХ КЛАССОВ	8
1.6	ПРИМЕР НА ЯЗЫКЕ PYTHON С ИСПОЛЬЗОВАНИЕМ ФУНКЦИЙ (ПРОЦЕДУРНЫЙ ПОДХОД)	10
1.7	ПРИМЕР НА ЯЗЫКЕ PYTHON С ИСПОЛЬЗОВАНИЕМ КЛАССОВ (ОБЪЕКТНО-ОРИЕНТИРОВАННЫЙ ПОДХОД)	11
1.8	ПРИМЕР НА ЯЗЫКЕ PYTHON С ИСПОЛЬЗОВАНИЕМ КОРТЕЖЕЙ (ОТЧАСТИ ФУНКЦИОНАЛЬНЫЙ ПОДХОД)	13
1.9	ПРИМЕР НА ЯЗЫКЕ PYTHON С ИСПОЛЬЗОВАНИЕМ NAMEDTUPLE (БОЛЕЕ ФУНКЦИОНАЛЬНЫЙ ПОДХОД)	15
1.10	ПРИМЕР НА ЯЗЫКЕ RUST	18
2	ВЫВОДЫ	20

1 Применение различных парадигм программирования на примере решения квадратного уравнения

Рассмотрим пример вычисления корней квадратного уравнения.

1.1 Пример на языке C# с использованием списка корней

Проект «SquareRoot». Файл: SquareRoot_Simple.cs

```
using System;
using System.Collections.Generic;

namespace SquareRoot
{
    /// <summary>
    /// Простое вычисление корней
    /// </summary>
    class SquareRoot_Simple
    {
        /// <summary>
        /// Вычисление корней
        /// </summary>
        public List<double> CalculateRoots(double a, double b, double c)
        {
            List<double> roots = new List<double>();
            double D = b * b - 4 * a * c;
            //Один корень
            if (D == 0)
            {
                double root = -b / (2 * a);
                roots.Add(root);
            }
            //Два корня
            else if (D > 0)
            {
                double sqrtD = Math.Sqrt(D);
                double root1 = (-b + sqrtD) / (2 * a);
                double root2 = (-b - sqrtD) / (2 * a);
                roots.Add(root1);
                roots.Add(root2);
            }
            return roots;
        }

        /// <summary>
        /// Вывод корней
        /// </summary>
        public void PrintRoots(double a, double b, double c)
        {
            List<double> roots = this.CalculateRoots(a, b, c);
            Console.WriteLine("Коэффициенты: a={0}, b={1}, c={2}. ", a, b, c);
            if(roots.Count == 0)
            {
                Console.WriteLine("Корней нет.");
            }
        }
    }
}
```

```

        else if (roots.Count == 1)
        {
            Console.WriteLine("Один корень {0}", roots[0]);
        }
        else if (roots.Count == 2)
        {
            Console.WriteLine("Два корня {0} и {1}", roots[0], roots[1]);
        }
    }
}
}

```

Корни возвращаются в виде списка.

Не очень надежно. Вдруг произошел сбой и вернулась пустая коллекция?

Для увеличения надежности будем использовать перечисление (enum).

1.2 Пример на языке C# с использованием перечисления

Проект «SquareRoot». Файл: SquareRoot_Enum.cs

```

using System;
using System.Collections.Generic;

namespace SquareRoot
{
    /// <summary>
    /// Перечисление для обозначения количества корней
    /// </summary>
    enum RootsEnum { NoRoots, OneRoot, TwoRoots }

    /// <summary>
    /// Вычисление корней с использованием перечисления
    /// </summary>
    class SquareRoot_Enum
    {
        /// <summary>
        /// Вычисление корней
        /// </summary>
        public void CalculateRoots(double a, double b, double c, out List<double>
roots, out RootsEnum rootFlag)
        {
            rootFlag = RootsEnum.NoRoots;
            roots = new List<double>();
            double D = b * b - 4 * a * c;
            //Один корень
            if (D == 0)
            {
                rootFlag = RootsEnum.OneRoot;
                double root = -b / (2 * a);
                roots.Add(root);
            }
            //Два корня
            else if (D > 0)
            {

```

```

        rootFlag = RootsEnum.TwoRoots;
        double sqrtD = Math.Sqrt(D);
        double root1 = (-b + sqrtD) / (2 * a);
        double root2 = (-b - sqrtD) / (2 * a);
        roots.Add(root1);
        roots.Add(root2);
    }
}

/// <summary>
/// Вывод корней
/// </summary>
public void PrintRoots(double a, double b, double c)
{
    List<double> roots;
    RootsEnum rootFlag;
    this.CalculateRoots(a, b, c, out roots, out rootFlag);
    Console.WriteLine("Коэффициенты: a={0}, b={1}, c={2}. ", a, b, c);
    if (rootFlag == RootsEnum.NoRoots)
    {
        Console.WriteLine("Корней нет.");
    }
    else if (rootFlag == RootsEnum.OneRoot)
    {
        Console.WriteLine("Один корень {0}", roots[0]);
    }
    else if (rootFlag == RootsEnum.TwoRoots)
    {
        Console.WriteLine("Два корня {0} и {1}", roots[0], roots[1]);
    }
}
}
}

```

Корни возвращаются в виде списка, а также возвращается значение перечисления, которое задает количество корней.

Не очень удобно. Список корней и значение перечисления возвращаются по отдельности. При возникновении ошибки они могут не соответствовать друг другу.

Можно ли их объединить? Теоретически их можно объединить в отдельный класс, но все равно список корней и значение перечисления можно изменить отдельно друг от друга.

В F# они объединены в единую структуру, которая называется алгебраическим типом.

1.3 Пример на языке F# с использованием алгебраического типа

Проект «SquareRootFSharp». Файл: Program.fs

```

//Для использования классов Math и Console
open System

// Алгебраический тип или "Discriminated Unions"
// Алгебраический тип - тип сумма из типов произведений
// | - означает "или" и задает тип-сумму
// * - означает "и" и задает произведение (кортеж, который соединяет все элементы)
// В абстрактных алгебрах наиболее близкой алгеброй является полукольцо

///Тип решения квадратного уравнения
type SquareRootResult =
    | NoRoots
    | OneRoot of double
    | TwoRoots of double * double //кортеж из двух double

///Функция вычисления корней уравнения
let CalculateRoots(a:double, b:double, c:double):SquareRootResult =
    let D = b*b - 4.0*a*c;
    if D < 0.0 then NoRoots
    else if D = 0.0 then
        let rt = -b / (2.0 * a)
        OneRoot rt
    else
        let sqrtD = Math.Sqrt(D)
        let rt1 = (-b + sqrtD) / (2.0 * a);
        let rt2 = (-b - sqrtD) / (2.0 * a);
        TwoRoots (rt1,rt2)

///Вывод корней (тип unit - аналог void)
let PrintRoots(a:double, b:double, c:double):unit =
    printf "Коэффициенты: a=%A, b=%A, c=%A. " a b c
    let root = CalculateRoots(a,b,c)
    //Оператор сопоставления с образцом
    let textResult =
        match root with
        | NoRoots -> "Корней нет"
        | OneRoot(rt) -> "Один корень " + rt.ToString()
        | TwoRoots(rt1,rt2) -> "Два корня " + rt1.ToString() + " и " + rt2.ToString()
    printfn "%s" textResult

[<EntryPoint>]
let main argv =
    //Тестовые данные
    //2 корня
    let a1 = 1.0;
    let b1 = 0.0;
    let c1 = -4.0;
    //1 корень
    let a2 = 1.0;
    let b2 = 0.0;
    let c2 = 0.0;
    //нет корней
    let a3 = 1.0;
    let b3 = 0.0;
    let c3 = 4.0;

    PrintRoots(a1,b1,c1)
    PrintRoots(a2,b2,c2)
    PrintRoots(a3,b3,c3)

    //|> ignore - перенаправление потока с игнорирование результата вычисления
    Console.ReadLine() |> ignore

```

0 // возвращение целочисленного кода выхода

У алгебраического типа есть недостаток. Его нельзя расширять в процессе реализации. А в C# можно воспользоваться решением на основе интерфейсов.

1.4 Пример на языке C# с использованием интерфейса и наследуемых классов

Проект «SquareRoot». Файл: SquareRoot_WithInterface.cs

```
using System;
using System.Collections.Generic;

namespace SquareRoot
{
    interface RootsResult { }
    class NoRoots : RootsResult { }
    class OneRoot : RootsResult
    {
        public double root { get; set; }
    }
    class TwoRoots : RootsResult
    {
        public double root1 { get; set; }
        public double root2 { get; set; }
    }

    /// <summary>
    /// Возможные варианты решения расширяются за счет использования интерфейса
    /// </summary>
    class SquareRoot_WithInterface
    {
        /// <summary>
        /// Вычисление корней
        /// </summary>
        public RootsResult CalculateRoots(double a, double b, double c)
        {
            List<double> roots = new List<double>();
            double D = b * b - 4 * a * c;
            //Один корень
            if (D == 0)
            {
                double rt = -b / (2 * a);
                return new OneRoot()
                {
                    root = rt
                };
            }
            //Два корня
            else if (D > 0)
            {
                double sqrtD = Math.Sqrt(D);
                double rt1 = (-b + sqrtD) / (2 * a);
                double rt2 = (-b - sqrtD) / (2 * a);
                return new TwoRoots()
                {
                    root1 = rt1,
```

```

        root2 = rt2
    };
}
//Нет корней
else
{
    return new NoRoots();
}
}

/// <summary>
/// Вывод корней
/// </summary>
public void PrintRoots(double a, double b, double c)
{
    RootsResult result = this.CalculateRoots(a, b, c);
    Console.Write("Коэффициенты: a={0}, b={1}, c={2}. ", a, b, c);
    string resultType = result.GetType().Name;
    if (resultType == "NoRoots")
    {
        Console.WriteLine("Корней нет.");
    }
    else if (resultType == "OneRoot")
    {
        OneRoot rt1 = (OneRoot)result;
        Console.WriteLine("Один корень {0}", rt1.root);
    }
    else if (resultType == "TwoRoots")
    {
        TwoRoots rt2 = (TwoRoots)result;
        Console.WriteLine("Два корня {0} и {1}", rt2.root1, rt2.root2);
    }
}
}
}

```

Этот вариант лучше, потому что он позволяет расширять варианты решения, добавляя новые классы, наследуемые от интерфейса. Но в F# тоже так можно.

1.5 Пример на языке F# с использованием интерфейса и наследуемых классов

Проект «SquareRootFSharpClass». Файл: Program.fs

```

open System

///Интерфейс
type SquareRootEmpty = interface end
//Наследуемые классы с вариантами решения
type NoRoots()=
    interface SquareRootEmpty
//Клсс содержит параметры, которые присваиваются свойству
type OneRoot(p:double)=
    interface SquareRootEmpty
    // Объявление свойства
    member val root = p : double with get, set

```



```

type TwoRoots(p1:double,p2:double)=
    interface SquareRootEmpty
        // Объявление свойства
        member val root1 = p1 : double with get, set
        member val root2 = p2 : double with get, set

    ///Функция вычисления корней уравнения
    let CalculateRoots(a:double, b:double, c:double):SquareRootEmpty =
        let D = b*b - 4.0*a*c;
        if D < 0.0 then (new NoRoots() :> SquareRootEmpty)
        else if D = 0.0 then
            let rt = -b / (2.0 * a)
            //Требуется явное приведение к интерфейсному типу
            (OneRoot(rt) :> SquareRootEmpty)
        else
            let sqrtD = Math.Sqrt(D)
            let rt1 = (-b + sqrtD) / (2.0 * a);
            let rt2 = (-b - sqrtD) / (2.0 * a);
            (TwoRoots(rt1,rt2) :> SquareRootEmpty)

    ///Вывод корней (тип unit - аналог void)
    let PrintRoots(a:double, b:double, c:double):unit =
        printf "Коэффициенты: a=%A, b=%A, c=%A. " a b c
        let root = CalculateRoots(a,b,c)
        //Оператор сопоставления с образцом по типу - :?
        let textResult =
            match root with
            | :? NoRoots -> "Корней нет"
            | :? OneRoot as r -> "Один корень " + r.root.ToString()
            | :? TwoRoots as r -> "Два корня " + r.root1.ToString() + " и " + r.root2.ToString()
            | _ -> "" // Если не выполняется ни один из предыдущих шаблонов
        printfn "%s" textResult

[<EntryPoint>]
let main argv =
    //Тестовые данные
    //2 корня
    let a1 = 1.0;
    let b1 = 0.0;
    let c1 = -4.0;
    //1 корень
    let a2 = 1.0;
    let b2 = 0.0;
    let c2 = 0.0;
    //нет корней
    let a3 = 1.0;
    let b3 = 0.0;
    let c3 = 4.0;

    PrintRoots(a1,b1,c1)
    PrintRoots(a2,b2,c2)
    PrintRoots(a3,b3,c3)

    //|> ignore - перенаправление потока с игнорирование результата вычисления
    Console.ReadLine() |> ignore
    0 // возвращение целочисленного кода выхода

```

Таким образом в F# можно использовать как «закрытые» алгебраические типы так и «открытую» к расширению реализацию на основе интерфейса и наследуемых классов.

1.6 Пример на языке Python с использованием функций (процедурный подход)

Проект «SquareRootPython». Файл: roots_proc.py

Реализован подход из SquareRoot_Simple.cs

Язык Python поддерживает процедурную, объектно-ориентированную, и, отчасти, функциональную парадигмы.

```
import sys
import math

def get_coef(index, prompt):
    """
    Читаем коэффициент из командной строки или вводим с клавиатуры

    Args:
        index (int): Номер параметра в командной строке
        prompt (str): Приглашение для ввода коэффициента

    Returns:
        float: Коэффициент квадратного уравнения
    """
    try:
        # Пробуем прочитать коэффициент из командной строки
        coef_str = sys.argv[index]
    except:
        # Вводим с клавиатуры
        print(prompt)
        coef_str = input()
    # Переводим строку в действительное число
    coef = float(coef_str)
    return coef

def get_roots(a, b, c):
    """
    Вычисление корней квадратного уравнения

    Args:
        a (float): коэффициент A
        b (float): коэффициент B
        c (float): коэффициент C

    Returns:
        list[float]: Список корней
    """
    result = []
```

```

D = b*b - 4*a*c
if D == 0.0:
    root = -b / (2.0*a)
    result.append(root)
elif D > 0.0:
    sqD = math.sqrt(D)
    root1 = (-b + sqD) / (2.0*a)
    root2 = (-b - sqD) / (2.0*a)
    result.append(root1)
    result.append(root2)
return result

def main():
    """
    Основная функция
    """
    a = get_coef(1, 'Введите коэффициент A:')
    b = get_coef(2, 'Введите коэффициент B:')
    c = get_coef(3, 'Введите коэффициент C:')
    # Вычисление корней
    roots = get_roots(a,b,c)
    # Вывод корней
    len_roots = len(roots)
    if len_roots == 0:
        print('Нет корней')
    elif len_roots == 1:
        print('Один корень: {}'.format(roots[0]))
    elif len_roots == 2:
        print('Два корня: {} и {}'.format(roots[0], roots[1]))

# Если сценарий запущен из командной строки
if __name__ == "__main__":
    main()

# Пример запуска
# qr.py 1 0 -4

```

1.7 Пример на языке Python с использованием классов (объектно-ориентированный подход)

Проект «SquareRootPython». Файл: roots_oop.py

Реализован подход из SquareRoot_Simple.cs, но с использованием класса.

```

import sys
import math

class SquareRoots:

    def __init__(self):
        """

```

```

Конструктор класса
'''

# Объявление коэффициентов
self.coef_A = 0.0
self.coef_B = 0.0
self.coef_C = 0.0
# Количество корней
self.num_roots = 0
# Список корней
self.roots_list = []

def get_coef(self, index, prompt):
    '''
    Читаем коэффициент из командной строки или вводим с клавиатуры
    Args:
        index (int): Номер параметра в командной строке
        prompt (str): Приглашение для ввода коэффициента
    Returns:
        float: Коэффициент квадратного уравнения
    '''
    try:
        # Пробуем прочитать коэффициент из командной строки
        coef_str = sys.argv[index]
    except:
        # Вводим с клавиатуры
        print(prompt)
        coef_str = input()
    # Переводим строку в действительное число
    coef = float(coef_str)
    return coef

def get_coefs(self):
    '''
    Чтение трех коэффициентов
    '''
    self.coef_A = self.get_coef(1, 'Введите коэффициент A:')
    self.coef_B = self.get_coef(2, 'Введите коэффициент B:')
    self.coef_C = self.get_coef(3, 'Введите коэффициент C:')

def calculate_roots(self):
    '''
    Вычисление корней квадратного уравнения
    '''
    a = self.coef_A
    b = self.coef_B
    c = self.coef_C
    # Вычисление дискриминанта и корней
    D = b*b - 4*a*c
    if D == 0.0:
        root = -b / (2.0*a)
        self.num_roots = 1
        self.roots_list.append(root)
    elif D > 0.0:
        sqD = math.sqrt(D)
        root1 = (-b + sqD) / (2.0*a)
        root2 = (-b - sqD) / (2.0*a)
        self.num_roots = 2
        self.roots_list.append(root1)
        self.roots_list.append(root2)

```

```

def print_roots(self):
    # Проверка отсутствия ошибок при вычислении корней
    if self.num_roots != len(self.roots_list):
        print(('Ошибка. Уравнение содержит {} действительных корней, ' + \
              'но было вычислено {} корней.').format(self.num_roots,
len(self.roots_list)))
    else:
        if self.num_roots == 0:
            print('Нет корней')
        elif self.num_roots == 1:
            print('Один корень: {}'.format(self.roots_list[0]))
        elif self.num_roots == 2:
            print('Два корня: {} и {}'.format(self.roots_list[0], \
self.roots_list[1]))

def main():
    '''
    Основная функция
    '''
    # Создание объекта класса
    r = SquareRoots()
    # Последовательный вызов необходимых методов
    r.get_coefs()
    r.calculate_roots()
    r.print_roots()

# Если сценарий запущен из командной строки
if __name__ == "__main__":
    main()

# Пример запуска
# roots_oop.py 1 0 -4

```

1.8 Пример на языке Python с использованием кортежей (отчасти функциональный подход)

Проект «SquareRootPython». Файл: roots_tuples.py

Реализован подход из «SquareRootFSharpClass», но вместо алгебраического типа используются кортежи.

Данный пример использует конструкцию match, которая появилась только в Python 3.10.

```

import sys
import math

```

```
def get_coef(index, prompt):
    '''
    Читаем коэффициент из командной строки или вводим с клавиатуры

    Args:
        index (int): Номер параметра в командной строке
        prompt (str): Приглашение для ввода коэффициента

    Returns:
        float: Коэффициент квадратного уравнения
    '''
    try:
        # Пробуем прочитать коэффициент из командной строки
        coef_str = sys.argv[index]
    except:
        # Вводим с клавиатуры
        print(prompt)
        coef_str = input()
    # Переводим строку в действительное число
    coef = float(coef_str)
    return coef


def get_roots(a, b, c):
    '''
    Вычисление корней квадратного уравнения

    Args:
        a (float): коэффициент A
        b (float): коэффициент B
        c (float): коэффициент C

    Returns:
        ... Список корней в форме кортежа
    '''
    D = b*b - 4*a*c
    if D == 0.0:
        root = -b / (2.0*a)
        return ('OneRoot', root)
    elif D > 0.0:
        sqD = math.sqrt(D)
        root1 = (-b + sqD) / (2.0*a)
        root2 = (-b - sqD) / (2.0*a)
        return ('TwoRoots', root1, root2)
    else:
        return ('NoRoots',)
```

```

def print_roots(roots_tuple):
    """
    Печать корней квадратного уравнения

    Args:
        Список корней в форме кортежа
    """
    match roots_tuple:
        case ('TwoRoots', root1, root2):
            print(f'Два корня: {root1} и {root2}')
        case ('OneRoot', root):
            print(f'Один корень: {root}')
        case ('NoRoots',):
            print('Нет корней')

def main():
    """
    Основная функция
    """
    a = get_coef(1, 'Введите коэффициент A:')
    b = get_coef(2, 'Введите коэффициент B:')
    c = get_coef(3, 'Введите коэффициент C:')
    # Вычисление корней
    roots = get_roots(a,b,c)
    # Вывод корней
    print_roots(roots)

# Если сценарий запущен из командной строки
if __name__ == "__main__":
    main()

# Примеры запуска
# python roots_tuple.py 1 0 -4
# Два корня: 2.0 и -2.0

# python roots_tuple.py 1 0 0
# Один корень: -0.0

# python roots_tuple.py 1 0 4
# Нет корней

```

1.9 Пример на языке Python с использованием NamedTuple (более функциональный подход)

Проект «SquareRootPython». Файл: roots_namedtuple.py

Реализован подход из «SquareRootFSharpClass», алгебраический тип эмулируется с использованием типа NamedTuple в языке Python.

Данный пример использует конструкцию `match`, которая появилась только в Python 3.10.

```
import sys
import math
from typing import NamedTuple

# Алгебраический тип
class SquareRootResult:
    NoRoots = NamedTuple("NoRoots", [])
    OneRoot = NamedTuple("OneRoot", [("root", float)])
    TwoRoots = NamedTuple("TwoRoots", [("root1", float), ("root2", float)])

def get_coef(index, prompt):
    """
    Читаем коэффициент из командной строки или вводим с клавиатуры

    Args:
        index (int): Номер параметра в командной строке
        prompt (str): Приглашение для ввода коэффициента

    Returns:
        float: Коэффициент квадратного уравнения
    """
    try:
        # Пробуем прочитать коэффициент из командной строки
        coef_str = sys.argv[index]
    except:
        # Вводим с клавиатуры
        print(prompt)
        coef_str = input()
    # Переводим строку в действительное число
    coef = float(coef_str)
    return coef

def get_roots(a, b, c):
    """
    Вычисление корней квадратного уравнения

    Args:
        a (float): коэффициент A
        b (float): коэффициент B
        c (float): коэффициент C

    Returns:
        Список корней в виде типа SquareRootResult
    """
```



```

'''
D = b*b - 4*a*c
if D == 0.0:
    root = -b / (2.0*a)
    return SquareRootResult.OneRoot(root)
elif D > 0.0:
    sqD = math.sqrt(D)
    root1 = (-b + sqD) / (2.0*a)
    root2 = (-b - sqD) / (2.0*a)
    return SquareRootResult.TwoRoots(root1, root2)
else:
    return SquareRootResult.NoRoots()

def print_roots(roots_tuple):
    '''
    Печать корней квадратного уравнения

    Args:
        Список корней в виде типа SquareRootResult
    '''
    match roots_tuple:
        case SquareRootResult.TwoRoots(root1, root2):
            print(f'Два корня: {root1} и {root2}')
        case SquareRootResult.OneRoot(root):
            print(f'Один корень: {root}')
        case SquareRootResult.NoRoots():
            print('Нет корней')

def main():
    '''
    Основная функция
    '''
    a = get_coef(1, 'Введите коэффициент A:')
    b = get_coef(2, 'Введите коэффициент B:')
    c = get_coef(3, 'Введите коэффициент C:')
    # Вычисление корней
    roots = get_roots(a,b,c)
    # Вывод корней
    print_roots(roots)

# Если сценарий запущен из командной строки
if __name__ == "__main__":
    main()

# Примеры запуска
# python roots_namedtuple.py 1 0 -4

```

```
# Два корня: 2.0 и -2.0

# python roots_namedtuple.py 1 0 0
# Один корень: -0.0

# python roots_namedtuple.py 1 0 4
# Нет корней
```

1.10 Пример на языке Rust

Проект «SquareRootRust». Файл: src/main.rs

Идея реализации напоминает «SquareRootFSharp».

```
use std::io;

#[derive(Debug, Copy, Clone)]
/// Тип решения квадратного уравнения
enum SquareRootResult {
    /// Unit-тип
    NoRoots,
    /// Один корень - кортежная структура
    OneRoot(f64),
    /// C-подобная структура
    TwoRoots { root1: f64, root2: f64 },
}

#[derive(Debug, Copy, Clone)]
/// Структура, соответствующая уравнению
struct Equation {
    /// Коэффициент A
    c_a: f64,
    /// Коэффициент B
    c_b: f64,
    /// Коэффициент C
    c_c: f64,
    /// Дискриминант
    disk: f64,
    /// Корни
    res: SquareRootResult,
}

impl Equation {
    /// Функция вычисления корней
    fn calculate_roots(&mut self) {
        self.disk = self.c_b.powi(2) - 4.0 * self.c_a * self.c_c;
        self.res = {
            if self.disk < 0.0 {
                SquareRootResult::NoRoots
            } else if self.disk == 0.0 {
```

```

        let rt = -self.c_b / (2.0 * self.c_a);
        SquareRootResult::OneRoot(rt)
    } else {
        let rt1 = (-self.c_b - self.diskr.sqrt()) / (2.0 * self.c_a);
        let rt2 = (-self.c_b + self.diskr.sqrt()) / (2.0 * self.c_a);
        SquareRootResult::TwoRoots {
            root1: rt1,
            root2: rt2,
        }
    }
};

}

/// Ввод одного коэффициента
fn get_coef(message: &str) -> f64 {
    return loop {
        let mut input = String::new();
        println!("{}", message);
        io::stdin()
            .read_line(&mut input)
            .expect("Неверно введена строка");
        match input.trim().parse() {
            Ok(val) => {
                break val;
            }
            Err(_) => {
                continue;
            }
        }
    };
}

fn get_coefs(&mut self) -> () {
    self.c_a = Equation::get_coef("Введите коэффициент A: ");
    self.c_b = Equation::get_coef("Введите коэффициент B: ");
    self.c_c = Equation::get_coef("Введите коэффициент C: ");
}

}

fn main() {
    use SquareRootResult::*;
    let mut eq = Equation {
        c_a: 0.0,
        c_b: 0.0,
        c_c: 0.0,
        disk: 0.0,
        res: SquareRootResult::NoRoots,
    };
    eq.get_coefs();
    eq.calculate_roots();
    let text_res = match eq.res {
        NoRoots => format!("Корней нет"),
    };
}

```

```

    OneRoot(rt) => format!("Один корень => {}", rt),
    TwoRoots { root1, root2 } => format!("Два корня => {} и {}", root1,
root2),
    };
    println!("{}", text_res);
}

```

2 Выводы

В настоящее время функциональные языки программирования широко используются для реализации различных задач.

Использование таких механизмов функционального программирования, как алгебраические типы и сопоставление с образцом, позволяют наглядно реализовать классические алгоритмы.